Krate

The Complete Guide

Everything about what it is, how it works, how it was built,
and how to answer any question about it

Krate Labs, Inc.

12 August 2026

Contents

Chapter 1

Read This First

1.1 What this document is

This is one document that contains everything about Krate: what it is, why it exists, how every piece works, how it was built, what went wrong along the way, what the numbers actually are, and how to answer any question anyone asks — a curious friend, a developer, an investor, or a hostile sceptic.

It assumes you know nothing. Not what WebAssembly is, not what Electron is, not what a runtime is. Every technical idea is explained with an example from ordinary life before any jargon appears.

The rule this document follows

Every number here was measured on 11 August 2026, or read directly out of the repository on that date. Where something is unknown, this document says it is unknown. Where a number depends on conditions, the conditions are stated. Nothing is estimated, rounded up, or assumed.

1.2 Krate in five sentences

1. An AI can now write you a working desktop app in a few minutes.
2. But you cannot easily *give* that app to anyone — they would need the right programming languages installed, or a 200-megabyte bundle, or they will see a scary security warning.
3. Krate turns that app into **one small file**, typically 15 to 30 kilobytes, that opens by double-click on macOS, Windows and Linux.
4. Before the app runs, it says in plain words what it wants to touch ("save files in its own private folder — never your files"), and the runtime *enforces* that, so an app cannot exceed what you allowed.
5. You send it like you would send a photo, or publish it and send a link.

In everyday terms

Think of a PDF. Before PDFs, sending someone a document meant hoping they had the same word processor, the same version, and the same fonts. The PDF made a document *one file that just opens*. Krate is trying to be the PDF of small applications.

Chapter 2

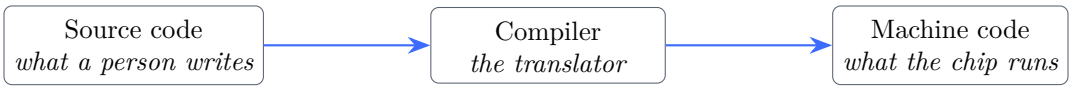
The Basics, Explained From Zero

This chapter assumes no technical knowledge at all. If you already know what WebAssembly and sandboxing are, you can skip to Chapter 3 — but this chapter is also the source of the analogies you should use when explaining Krate to other people.

2.1 What is a program, really?

A program is a list of instructions for a machine. But a computer's processor does not understand English, or Python, or any language a person writes in. It understands only **machine code** — numbers that mean "add these", "store this here", "jump there".

So there is always a translation step:



2.1.1 Why the same app does not run everywhere

Here is the problem that has existed since computers began: **machine code is different on every kind of computer.**

A Mac with an Apple chip speaks one dialect (called `arm64`). An older Mac or most Windows PCs speak another (`x86_64`). And even on the same chip, macOS, Windows and Linux each expect programs to be packaged differently and to ask for files and windows in different ways.

In everyday terms

It is like electrical plugs. The electricity is the same idea everywhere, but the plug shape is different in the UK, the US and India. A device built for one socket physically cannot go into another.

So historically, if you wrote an app and wanted everyone to run it, you had to build it three times, test it three times, and ship three different files. That is expensive, and it is why most small useful programs never get shared at all.

2.2 The four ways people have solved this, and what each costs

2.2.1 Way 1: Build it separately for each system ("native")

Write the app, then compile it once for Mac, once for Windows, once for Linux. Three builds, three files, three sets of bugs.

- **Good:** fastest possible, feels exactly like the system it runs on.

- **Bad:** three times the work, forever. And the person receiving it still gets security warnings, because the operating system does not know who you are.

2.2.2 Way 2: Ship a whole web browser with your app ("Electron")

This is what Slack, Discord, VS Code and Notion do. The app is really a web page, and the "app" you download contains an entire copy of the Chrome browser to display that web page.

- **Good:** write once, runs everywhere, and web developers can build it.
- **Bad:** enormous. A tool that does something simple arrives as 80 to 200 megabytes, uses a lot of memory, and starts slowly.

In everyday terms

It is like mailing someone a letter, but because you are not sure they own reading glasses, you post them a pair of glasses with every letter. It works, but the envelope is now enormous, and they now have forty pairs of glasses.

2.2.3 Way 3: Make it a website instead

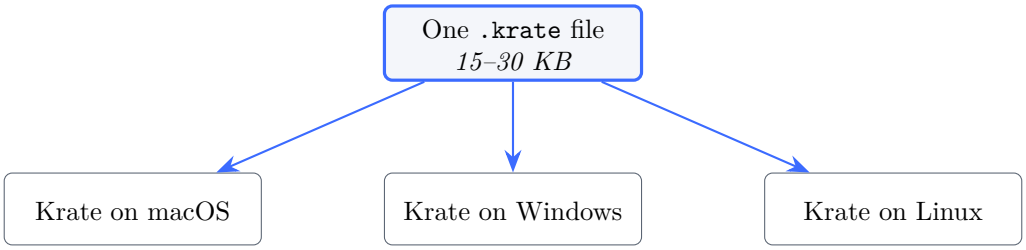
Do not ship anything; put it on the internet and send a link.

- **Good:** nothing to install, works on everything.
- **Bad:** you do not own it, it needs internet, it cannot easily touch your own files, and it disappears the day someone stops paying the hosting bill.

2.2.4 Way 4: A shared runtime (this is where Krate lives)

Ship the app in *one* portable format, and put a small program on each computer that knows how to run that format.

This idea is old and proven: it is how Java worked, and how a PDF reader works. The app file is the same everywhere; the thing that reads it is built for each system.



2.3 What is WebAssembly?

WebAssembly (usually written **Wasm**) is a portable form of machine code. It was invented so that web browsers could run fast programs written in languages other than JavaScript, but it turned out to be useful far beyond browsers.

Three properties matter for Krate:

1. **It is portable.** One Wasm file runs on any chip and any operating system that has a Wasm runtime.
2. **It is fast.** It is real compiled code, not an interpreted script. It runs at close to native speed.

3. **It is sandboxed by design.** This is the crucial one. A Wasm program *cannot do anything at all* on its own — it cannot open a file, reach the network, or draw on the screen. It can only do what the runtime around it explicitly hands over.

In everyday terms

A normal program is a guest with the run of your house. A Wasm program is a guest in a room with no doors — the only things it can reach are what you deliberately pass through a hatch. That is not a restriction someone added afterwards; it is how the room is built.

That third property is why Krate can make a promise no ordinary app can make. The permission wall is not politeness. The app genuinely has no way to reach anything you did not grant.

2.4 What is a "runtime"?

A runtime is the program that runs other programs.

When you double-click a `.krate` file, you are not running the app directly — your operating system runs **Krate**, and Krate reads the app and runs it inside itself, deciding what the app is allowed to do.

In everyday terms

A runtime is a games console. The disc holds the game, but the disc cannot do anything by itself. The console reads it, runs it, and controls what the game can reach — the screen, the controller, the memory card. The same disc works in any console of that type, anywhere in the world.

So what is "the Krate runtime"?

It is a single program, about 21 megabytes, that you install once. It contains: a WebAssembly engine (`wasmtime`), a permission checker, a drawing system that can paint windows on three operating systems, and everything else an app might legitimately need — a clock, a filesystem gate, a network gate, a key-value store, a database, sound, and text rendering.

Chapter 3

What Krate Actually Is

3.1 The one-sentence version

Krate

Krate makes an AI-written app into one small file that runs on any computer, tells you what it wants before it runs, and cannot do anything you did not allow.

3.2 The problem, stated precisely

The making of software just got solved. Anyone can now ask an AI for a tool and get working code in minutes. But the *delivering* of software did not change at all:

- If your AI writes Python, the person you send it to needs Python installed, and the right version, and the libraries.
- If it writes a web app, someone must host it, forever.
- If it writes a native app, it must be built three times, and it will still trigger security warnings on arrival.
- If it wraps a browser (Electron), a two-hundred-line tool becomes a two-hundred-megabyte download.

And beneath all of that is a trust problem: **why would you run a program a stranger's AI wrote?** You cannot read the code. You have no idea what it touches.

3.3 The three things Krate does about it

3.3.1 One file that runs anywhere

A Krate app is one `.krate` file. Measured examples from the live gallery on 11 August 2026:

App	Size
"Sadas" (published by a user)	14,974 bytes
"Add dvd screensaver"	15,135 bytes
"World Clock"	24,261 bytes
"Journal"	29,878 bytes
Breakout game (playable, in-repo)	31,455 bytes

For comparison, the same kind of tool built with Electron is typically 80–200 megabytes. A 30-kilobyte game is roughly **five thousand times smaller** than a 150-megabyte Electron app.

3.3.2 A permission wall that is real

Before an app runs, the person sees what it wants, in the app's own words:

```
Krategram wants to:

  Open the feed window                    [on]
    ui.window:create

  Print progress markers                  [on]
    io.stdout

         [ Open ]        Cancel
```

Two things make this different from the permission prompts you are used to:

1. **Denying still opens the app.** Turn something off and the app runs without that power, rather than refusing to start.
2. **It is enforced by the machine, not by policy.** The app physically cannot call what it was not granted — the check happens at the boundary between the app and the runtime, on every single call.

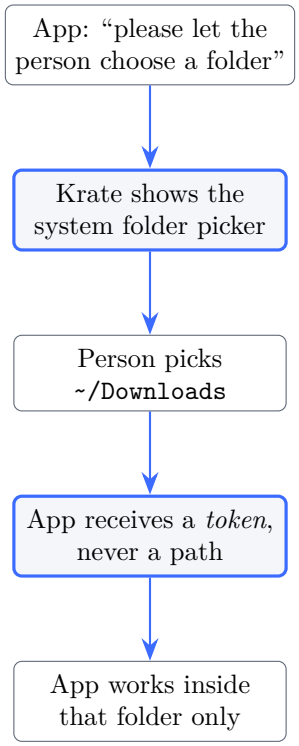
3.3.3 The AI writes it, and a machine checks the AI's work

You type what you want in plain words. Whichever AI you already pay for (Claude, Codex, Gemini, Copilot, Grok) writes real compiled code. Then Krate's own checker — `check-app` — builds it, verifies it only uses Krate's APIs, runs it, resizes its window, clicks it, and watches whether it closes itself. Only then do you get a file.

3.4 The idea that impresses engineers most: "the pick is the grant"

Consider an app that tidies your Downloads folder. Every other system solves this by asking for broad filesystem access — and then you are trusting it.

In Krate, such an app **cannot name a folder at all**. There is no capability it can declare that would let it. Instead it asks you to pick one. Your pick becomes the permission: scoped to that folder, for that run only, and gone when the app closes.



The working example ships in the repository as `apps/krate-tidy`. Its manifest declares **zero** filesystem capabilities.

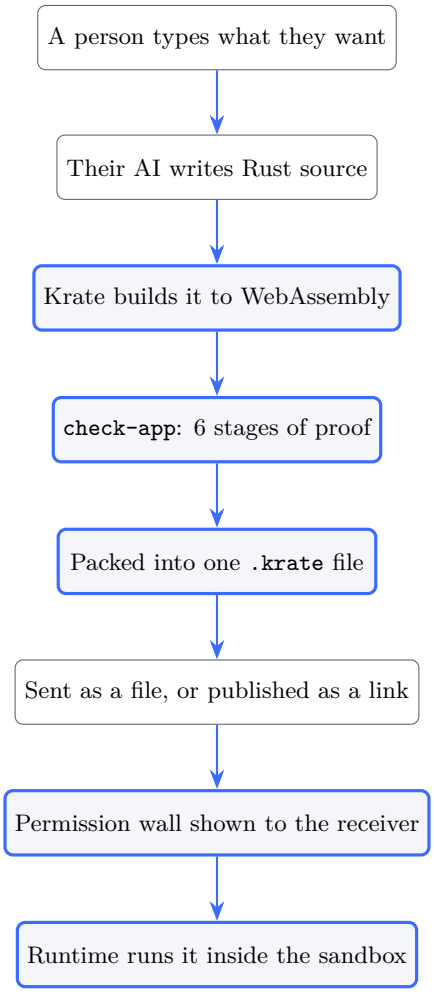
In everyday terms

It is the difference between giving a builder the keys to your house, and walking them to one room and standing there while they work.

Chapter 4

How Krate Works Inside

4.1 The whole system on one page



4.2 How big is this, really?

There are several honest answers to “how much code is Krate”, and they differ by a factor of eight. Knowing which is which matters, because the biggest number is the one a sceptic will check.

Measure	Lines	What it means
Lines added, all commits	895,988	What GitHub's contribution graph counts
Lines deleted, all commits	68,152	
Net	827,836	Added minus deleted
All tracked files today	921,897	Everything in the repository
of which generated	618,821	`bindings.rs`, written by a machine
hand-written Rust	**112,831**	**The real engineering number**
documentation (Markdown)	46,910	182 files
shell scripts	9,498	111 files
web (HTML/TS/JSON)	13,077	
interface definitions (WIT)	2,461	23 files
CI configuration (YAML)	2,763	6 workflows

Use the right number, or lose the argument

"Nearly 900,000 lines" is true and it is what GitHub shows — but **618,821 lines of it is generated** `bindings.rs`, the glue the toolchain writes automatically from the interface definitions. Quote the big number without that caveat and the first person who runs `wc -l` destroys your credibility.

The number that survives every check is **112,831 lines of hand-written Rust**, plus 46,910 lines of documentation, built between April and August 2026. For systems code — a WebAssembly runtime, three desktop windowing backends, two phone players, a renderer, a package format, and a cloud — that is a serious body of work, and it is one you can defend line by line.

4.2.1 What that hand-written code is spread across

Area	Hand-written lines
Runtime, CLI, adapters, tooling (`crates/`)	90,888
Example applications (`apps/`)	20,853
Total	**112,831**

Two files carry an unusual share and are worth knowing by name: the CLI's `main.rs` is 10,177 lines, and the runtime's GUI host (`phase3_gui_host.rs`) is 4,320. The GUI host is the piece that turns one portable app into a real window on any operating system, which is why it is the second-largest single file in the project.

4.3 What is inside the runtime

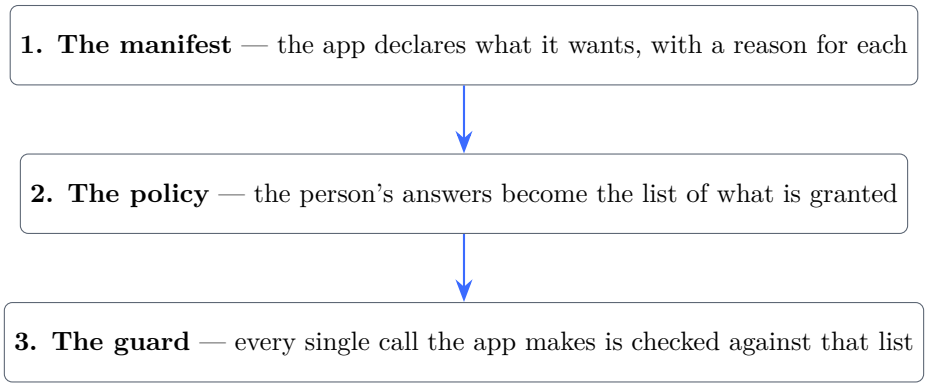
Krate is 112,831 lines of hand-written Rust across 20 components (see Section 4.2 for how that relates to the much larger total line count on GitHub). The largest pieces, measured on 11 August 2026:

Component	Lines	What it does
`cli`	23,764	The `krate` command: run, create, check, pack, publish, doctor

Component	Lines	What it does
`runtime`	22,483	Runs apps, enforces permissions, draws canvases
`tools`	8,077	Internal development and verification tooling
`adapter-common`	8,032	Shared drawing and text rendering
`bindings-rust`	6,534	The SDK an app is written against
`adapter-macos`	5,598	Native windows on macOS
`adapter-ios`	3,109	iPhone windows and GPU rendering
`author`	2,723	Drives the AI that writes apps
`port`	2,644	Analyses existing projects for portability
`mcp`	2,633	Lets a chat model build apps by talking
`adapter-android`	2,258	Android windows and touch
`adapter-windows`	2,008	Native windows on Windows
`adapter-linux`	1,987	Native windows on Linux
`layout`	1,393	Arranges things on screen
`bundle`	1,385	Reads and writes the `.krate` file format
`manifest`	1,212	Parses and validates what an app asks for
`krate-hub`	513	The cloud service
`policy`	494	Decides what is granted
`player-android`	420	The Android app that opens `.krate` files
`player-ios`	240	The iPhone app that opens `.krate` files

4.4 How the permission wall actually works

There are three layers, and all three must agree. This matters, because we learned the hard way that two out of three is the same as zero.



In August 2026 we found (bug K-086) that the third layer was missing for dialog boxes: the wall showed the words, the policy recorded the answer, and nothing checked it. Any app could open file pickers it had never asked for. All three layers now carry their own test.

4.5 The six checks every app must pass

`check-app` is the machine that decides whether an app is real. An AI writes code, this decides whether the code is acceptable, and the AI tries again until it passes.

	Stage	What it proves
1	layout	The three required files exist and the code is not obviously wrong
2	manifest	Everything it asks for is real, scoped, and actually used by the code
3	build	It compiles to WebAssembly
4	imports	It uses *only* Krate APIs — no smuggled operating-system access
5	run	It actually runs, headless, without crashing
6	usability	Its window opens, survives a resize, responds to a click, and does not close itself

Stage 6 is unusual and it is the one that catches what people actually complain about. On 11 August 2026 it caught 21 of our own 32 example apps drawing at a fixed size and ignoring the window — the exact bug a first-time user reported from Windows.

Chapter 5

The Engineering, In Depth

The previous chapter showed the shape of the system. This one goes through the five hardest problems inside it and how each was solved. If someone technical wants to know whether there is real engineering here, this is the chapter to walk them through.

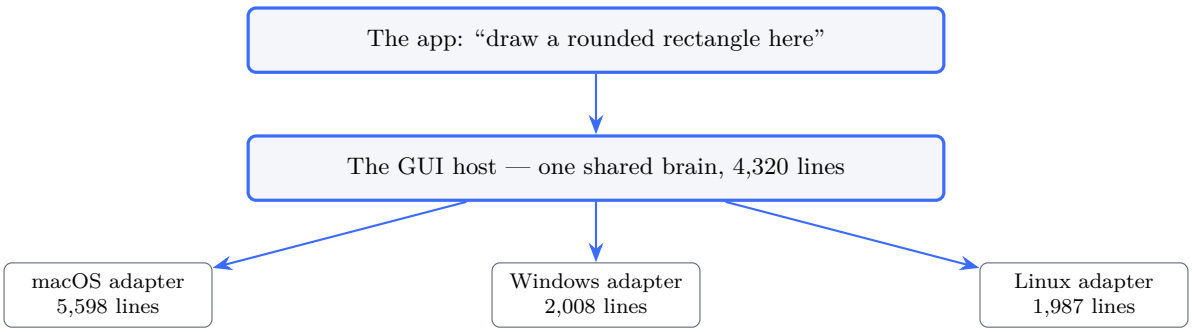
5.1 Problem 1: one app, three completely different windowing systems

macOS, Windows and Linux do not merely differ in details — they disagree about what a window *is*. macOS wants Objective-C objects and a run loop that owns the main thread. Windows wants a message pump. Linux wants you to talk to a display server that might be one of two incompatible kinds.

In everyday terms

It is like being asked to write one set of driving instructions that works in a country that drives on the left, one that drives on the right, and one where the roads are numbered backwards. You cannot write one set of turns. You have to write directions in terms of *intent* — "go towards the river" — and have a local translate.

Krate's answer is the **adapter**: one small piece of code per operating system that knows only how to translate intent into that system's native calls.



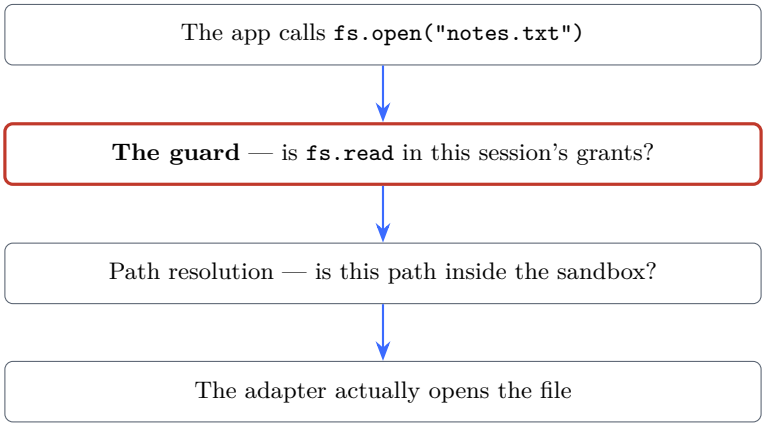
The important design choice is where the line sits. Everything an app can observe — layout, drawing, event order — lives *above* the line, in the shared host, so it is identical everywhere by construction. Only the irreducible platform differences live below it. That is why the adapters are small: 2,008 lines for Windows against 4,320 for the shared host.

The rule is machine-enforced, not a convention

A tool called `check-adapter-boundary` runs in CI and reads the adapter source files to verify that they have not started making decisions that belong above the line. There are **21** such tools in `crates/tools/src/bin/`, each policing one architectural rule — interface parity, widget parity, component imports, UAPI freeze locks.
Most projects enforce behaviour with tests. This one also enforces its own *architecture*, automatically, on every change.

5.2 Problem 2: making the sandbox real at the call boundary

The permission wall is only as good as the place it is checked. Krate's check happens at what the source calls "the narrow waist": one dispatcher that every single call from an app must pass through.



There is no way around it, because the app has no other way to reach the outside. It cannot call the operating system directly — WebAssembly gives it no instruction that could.

5.2.1 What the path check actually rejects

An app that has been granted file access still cannot wander. Before any path is used, it is rejected if it:

- is absolute (`/etc/passwd`, `C:\Windows`);
- contains a backslash at all, which stops Windows-style escapes;
- contains any control character;
- contains *any* component that is not an ordinary name — which is what rules out `..`, the classic "climb out of the folder" trick;
- is empty.

Only a path made entirely of ordinary names, resolved underneath the app's own directory, survives. And `~` (your home folder) is refused when the manifest is parsed, before the app ever runs.

Why the file picker is the interesting case

If an app cannot name a path, how does it open *your* file?
It doesn't. You pick the file, and Krate hands the app a **token** — an opaque string that means "the thing they chose". The app calls `open_chosen(token)`. It never learns where the file is, and the token store lives and dies with the run, so a token from yesterday resolves to nothing today.

5.3 Problem 3: drawing the same picture on every machine

An app says "fill a rounded rectangle". Something has to turn that into coloured pixels — identically on a Mac, a PC, a Linux box, and a phone, or the same app looks different to different people.

Krate does this in software, on the CPU, using a vector renderer (`vello_cpu`) with real text shaping (`parley`, `harfrust`, `skrifa`). Software rendering sounds like the slow choice, and it is a deliberate trade:

	CPU rendering (chosen)	**GPU rendering**
Same pixels everywhere	Yes, by construction	No — drivers differ
Works on every machine	Yes	Needs a working driver
Speed	≈400M pixels/second	Faster
Memory	Flat, about 80 MB	More

For the kind of app Krate is for, 400 million pixels a second is far more than enough — a full 4K frame is about 8 million pixels. The GPU path was added on iPhone in August 2026 where it matters more, and was checked against the CPU path rather than assumed to match.

5.3.1 The three coordinate systems, and why they exist

This trips up everyone, so it is worth stating plainly. A canvas has three different sizes at once:

Name	**What it is**
Design size	The made-up coordinates the app draws in, if it chose to have one — say "my world is 800 by 600"
Logical size	The window's size in the units a person thinks in
Physical size	The actual pixel buffer, which on a high-resolution screen is two or three times bigger

In everyday terms

A design size is like drawing a poster at A3 and letting the print shop scale it to whatever paper you bought. The proportions never distort, and any leftover paper becomes an even border.

The app declares a design size with `set-design-size`, and the runtime scales it uniformly to the window and centres it, so the app never distorts. Crucially, pointer and scroll events are converted *back* into design coordinates before the app sees them, so a click lands where it looks like it landed. Getting that conversion wrong in one direction is the bug class that makes an app look fine but be unclickable.

5.4 Problem 4: the iPhone forbids the normal fast path

A WebAssembly runtime is normally fast because it compiles the app into real machine code as it loads — "just-in-time" compilation. Apple does not allow an app to do this on an iPhone. On paper, that rules Krate out of iOS entirely.

The answer was to switch engines rather than fight the rule: on iOS the runtime uses **Pulley**, wasmtime's portable interpreter, which executes the app without ever generating machine code. It is slower per instruction, but it is legal, and it was checked to produce *pixel-identical* output to the compiled path rather than assumed to.

Why this is a good answer and not a compromise

The alternative — shipping a separate, hand-written iOS version of every app — would have destroyed the entire premise, which is that one file runs everywhere. Taking a speed penalty on one platform to keep the promise intact is the right trade, and it is testable: the same file, the same pixels, one slower engine.

5.5 Problem 5: apps must not be able to smuggle in OS access

This is the subtlest problem in the project, and the one most likely to be asked about by someone who knows WebAssembly.

An app is written in Rust. Ordinary Rust code uses the standard library, and the standard library assumes an operating system underneath — so the moment an app calls something innocuous, the compiler quietly adds `wasi:*` imports (requests for operating-system services) that Krate never agreed to provide.

In everyday terms

You hire a contractor who promises to work only with tools you hand them. But their toolbox has a hidden compartment that automatically packs a crowbar whenever they bring a hammer. Nobody is being dishonest — the toolbox just works that way.

The fix is to build apps `#![no_std]`: without the standard library, with Krate's SDK supplying the few things that genuinely need to exist (memory allocation, panic handling). Then the only imports in the finished app are `krate:*` ones.

And this is checked, not trusted: stage 4 of `check-app` reads the compiled component's import list and fails it if anything other than `krate:*` appears. When it fails, the error maps the leaked import back to the discipline that removes it.

The measured history of this problem

When eight apps were first authored by an AI, **three failed** for exactly this reason — and the cause was not obvious code. One was `Vec::with_capacity`, whose allocation-failure handler dragged in the whole operating-system surface. Another surfaced only when a database query returned a list.

Each was tracked down to a specific line and fixed at the SDK level, so that app authors never have to think about it. This is the kind of problem that does not appear in a demo and takes weeks to find in production.

Chapter 6

The Complete Technical Reference

This chapter is the deep detail: the exact capability list, the exact file format, the exact commands, and the exact release pipeline. You do not need to memorise it. You need to know it exists so that when somebody asks a specific question, the answer is here.

6.1 The capability list, complete

There are exactly **37** capabilities. **14** are granted to every app automatically; **23** must be explicitly asked for, appear on the permission wall, and can be refused.

The dividing principle is simple and worth being able to state: *a capability is granted by default only if using it moves no data and takes no outward action.* Reading the clock moves no data. Reading your clipboard does. Showing a message box takes no outward action. Sending a desktop notification does.

6.1.1 Granted to every app (14)

Capability	Phase	Why it needs no permission
`io.stdin`	2	Text in and out of the app's own terminal
`io.stdout`	2	
`io.stderr`	2	
`io.args`	2	The arguments it was started with
`io.log`	2	Its own diagnostic output
`time.clock`	2	What time it is reveals nothing
`time.monotonic`	2	How long something took
`time.sleep`	2	Waiting
`locale.info`	2	Your language and region
`locale.format`	2	Formatting a date the local way
`ui.window:create`	3	An app with a window is the normal case
`ui.dialog:message`	3	Shows text; no data moves
`ui.dialog:confirm`	3	Takes a click; no data moves
`gfx.gpu:basic`	3	Drawing inside its own window

6.1.2 Must be asked for (23)

Capability	Phase	What it lets the app do
`fs.read:<glob>`	2	Read files matching a pattern
`fs.write:<glob>`	2	Write files matching a pattern
`fs.list:<glob>`	2	See what files exist
`fs.remove:<glob>`	2	Delete files
`fs.mkdir:<glob>`	2	Create folders
`net.connect:<host>:<port>`	2	Reach one named host and port — not "the internet"
`store.kv`	2	Remember things between runs
`store.sql`	2	Keep its own database
`store.secret`	2	Keep tokens and keys, encrypted at rest
`random.bytes`	2	Draw random numbers
`ui.clipboard:read`	3	Read what you copied
`ui.clipboard:write`	3	Put something on your clipboard
`ui.menu:system`	3	Add to the system menu bar
`ui.open-url`	3	Hand a link to your browser
`ui.notify`	3	Send a desktop notification
`ui.dropzone:<mime>`	3	Accept files dragged onto it
`ui.dialog:file-open`	3	Ask you to pick a file to open
`ui.dialog:file-save`	3	Ask you where to save
`ui.dialog:open-folder`	3	Ask you to pick a folder — the biggest dialog grant
`ui.dialog:*`	3	All dialogs at once
`gfx.gpu:compute`	3	General-purpose GPU work (not implemented yet)
`audio.playback`	3	Make sound
`audio.capture`	3	Use your microphone

Two details that show the design is serious

Network access is not "the internet". An app cannot ask to go online. It asks for one host and one port — `net.connect:api.example.com:443`. A weather app that asks for the weather service cannot quietly talk to anywhere else.

`ui.dialog:*` is not granted by default, and that was a bug fix. Message and confirm boxes are default-granted, and for a while the wildcard was too — which silently covered file-open, file-save and open-folder, making their "explicit ask" meaningless. That was bug K-086. The wildcard is now an explicit ask.

6.2 What a manifest looks like

This is the real manifest from `apps/krate-tidy` — the folder-tidying app from Chapter 3. It is worth reading closely because of what is *absent*.

```
[app]
id = "dev.krate.tidy"
name = "Tidy"
version = "0.1.0-dev"
```

```
entry = "target/wasm32-wasip1/release/krate_tidy.wasm"
world = "krate:app/gui@0.2.0"

[[capabilities]]
cap = "ui.window:create"
rationale = "Open the tidy window"
required = true

[[capabilities]]
cap = "ui.dialog:open-folder"
rationale = "Ask you to pick the folder to tidy -- it can touch only
             what you pick, this run"
required = true
```

An app whose entire job is reorganising a folder full of your files declares **no filesystem capability at all**. Every `rationale` line is what the person actually reads on the wall, which is why they are written in plain words rather than technical ones.

6.3 Inside the .krate file

A `.krate` is a zip archive with a fixed shape:

```
myapp.krate
  |-- manifest.toml     what it is and what it wants
  |-- code.wasm         the compiled component
  `-- assets/           optional images, sounds, fonts
```

The format is deliberately boring, but four rules in it are defensive and worth being able to name:

1. **Entry names are matched exactly.** The manifest and component must be called exactly those names.
2. **Asset paths accept only ordinary relative components under `assets/`**, so a crafted archive cannot write outside the bundle (the "path traversal" attack).
3. **Both the archive and each decompressed entry are size capped**, which bounds the classic "zip bomb" — a small file that expands to fill your disk.
4. **The manifest's declared entry must match the component actually inside**, so a bundle cannot claim to be one thing and carry another.

Krate identifies a bundle by looking for `manifest.toml` as the first entry, which is how it can tell a `.krate` from a bare `.wasm` file without parsing the whole thing.

6.4 The interfaces an app can call

An app talks to the runtime through typed interfaces written in WIT (the WebAssembly Interface Types language). Twelve exist, with 125 functions between them:

Interface	Funcs	What it covers
`gfx`	35	Drawing: shapes, images, text, canvases, clipping
`ui`	29	Windows, events, dialogs, menus, clipboard
`fs`	12	Files, inside the sandbox
`store`	12	Key-value, SQL, and secrets
`audio`	11	Playback and capture
`io`	10	Terminal in and out, arguments, logging
`locale`	4	Language, region, formatting
`random`	3	Random bytes
`speech`	3	Speech recognition
`time`	3	Clock, monotonic time, sleep
`net`	2	Connecting to a named host
`resources`	1	Bundled assets

Why this list is the honest measure of what Krate can do

An app can do exactly these 125 things and nothing else. That is the whole surface. When somebody asks "can a Krate app do X?", the answer is determined by this table — and when the answer is no, that is a `runtime-hole` on the bug board, which is the class we prioritise.

6.5 The commands

Command	What it does
`krate run`	Run an app
`krate create`	Ask an AI for an app and get a finished `.krate`
`krate check-app`	The six-stage oracle: build, import-check, run, judge
`krate pack`	Bundle a component and manifest into one file
`krate publish`	Upload it and print a link anyone can run
`krate doctor`	Check this machine's setup
`krate ai`	Show which AI coding tools are installed
`krate connect`	Set up Claude Desktop or Cursor to build Krate apps
`krate mcp`	Run the server that lets a chat model build apps by talking
`krate krate-mode`	Print the prompt that teaches any chat model to write Krate apps
`krate authoring-context`	Write the context pack for an app directory
`krate manifest`	Inspect and validate a manifest
`krate report`	Analyse an existing project for portability (read-only)
`krate port`	Port an existing project
`krate telemetry`	Turn the anonymous usage count on or off
`krate version`	Print version information

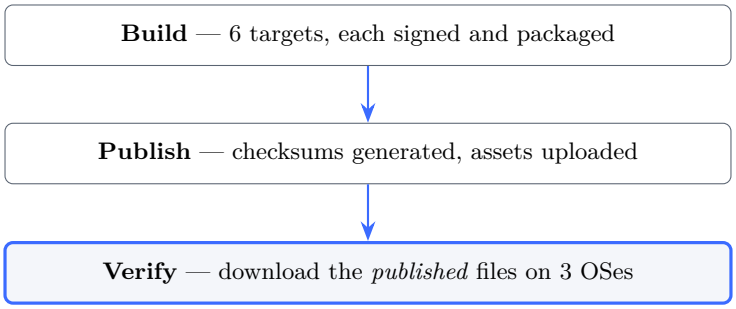
Two of these are worth understanding as strategy rather than features. `krate connect` and `krate mcp` mean a person never has to use a terminal at all — they talk to Claude Desktop or

Cursor, and the app appears. And `krate report` is deliberately read-only: it tells you whether an existing project could become a Krate app without touching it.

6.6 The release pipeline

This is the part that answers "how do I know a release actually works?"

Every release builds on six platforms, publishes, and then a *separate set of jobs on three operating systems downloads what was published* and proves things about it:



The verify stage checks, on the real downloaded files:

- the checksum a careful person would check actually matches;
- the archive unpacks into a folder, not files with backslashes in their names (this check exists because we shipped that bug — see Chapter 6);
- the binary reports the version on the tin;
- `cargo-component` ships beside it, so the build path works;
- the Windows document icon is present (K-058 shipped without it);
- macOS signing, entitlements and Gatekeeper pass (K-063, K-068);
- an app *builds with the published binary and runs headless*;
- **the permission wall still refuses without grants.**

The sentence to use when someone doubts the quality

"Every release downloads its own published files on three operating systems and re-proves eight things about them, including that the permission wall still refuses. Most of those checks exist because we shipped that exact bug once."

Chapter 7

How the Quality Is Actually Enforced

"How do I know this works?" deserves a better answer than "we tested it". Here is the whole apparatus, in layers.

7.1 The five layers

Layer	What it catches	Size
Unit and integration tests	Ordinary defects	1,141 tests
Architecture tools	Design rules being broken	21 tools
Nightly fuzzing	Crashes on hostile input	3 targets
`check-app`	Apps that are valid but unusable	6 stages
Post-publish verification	Broken releases	3 operating systems

Each layer catches something the others structurally cannot. That is the point of having five.

7.2 Fuzzing: throwing garbage at the parsers

Every night, a machine spends 80 minutes per target feeding randomly corrupted input into the three most security-sensitive parsers in the project, looking for a crash:

Target	Why this one
`manifest_parse`	The manifest is the first thing read from an untrusted file
`policy_match`	Deciding what is granted must not be trickable
`logical_path_parse`	Path handling is where sandbox escapes live

In everyday terms

Fuzzing is hiring someone to mash the keyboard at your app forever and tell you if it ever falls over. It is unglamorous and it finds the crashes that careful thinking misses, because it does not know what is "reasonable" to try.

Those three were not chosen at random — they are exactly the code paths that read data an attacker controls.

7.3 `check-app`, stage by stage

This is the machine that decides whether an app is real. It matters more than the tests, because it is what stands between an AI's output and a person's screen.

	Stage	What it proves	Catches
1	layout	The required files exist and the code is not obviously wrong	Missing pieces
2	manifest	Everything asked for is real, scoped, and actually used	Over-asking
3	build	It compiles to WebAssembly	Broken code
4	imports	It uses only `krate:*`	Smuggled OS access
5	run	It runs headless without crashing	Crashes
6	usability	Window opens, survives a resize, answers a click, stays open	Unusable apps

7.3.1 Why stage 6 exists, and what it does

Stages 1–5 are what any careful project would build. Stage 6 is the unusual one, and it exists because six passing stages once approved an app that could not be clicked.

The driver behaves like an impatient person: it lets the app settle, grows the window, presses the middle of it, watches for fifteen seconds, and records what happened.

The design rule that makes it trustworthy

Observe, never require. The driver reports what it saw. Anything it could not measure is recorded as "not observed" — never as a failure.
This sounds weak and is the opposite. A gate that fails apps it merely failed to measure gets switched off within a week, and a switched-off gate protects nothing. By refusing to guess, this one stays on.

Two limits are stated in the source rather than hidden, and you should state them too if asked:

- **Resize measures the canvas, not the picture.** Nothing in a painted frame distinguishes a layout that re-flowed from one that was merely stretched. So the check asks whether the app's canvas followed the window at all.
- **A press on a canvas app is a guess.** A canvas app draws its own buttons, so the host presses the middle and hopes. Landing on empty space is indistinguishable from a dead button, so that case is reported as unobserved rather than failed.

7.4 What this apparatus actually caught

Not hypothetical. On 11 August 2026, stage 6 caught **21 of our own 32 example apps** drawing at a fixed size and ignoring the window — the same defect a first-time user had reported from Windows. The fleet went from 19 passing to 32 passing in one day.

And what it did not catch

The same week, a scripted fix gave seventeen apps a *square* design size through a bad regular expression, and it shipped in v0.1.10. The gate could not see it, because a square design size is a perfectly valid design size.
A gate checks the property you thought to encode. This is why the outsider testing in Chapter 6 exists alongside it, and why "it passed CI" is never the last word.

Chapter 8

The Numbers

Every number in this chapter was measured on 11 August 2026 on an M-series Mac, with the machine otherwise idle. Conditions are stated because they change the answer.

8.1 Speed

Measurement	Result	What it means
Krate app launch, end to end	11.3 ms	From typing the command to the app finishing
The same program, native binary	5.0 ms	Compiled directly for this Mac
Ratio	2.25×	Krate is about twice as slow to *start*
Runtime: compile + start a component	3.3 ms	The engine's own work
Runtime: steady-state work	61 μs	Once loaded, per invocation

Do not claim Krate is faster than native

It is not. It is roughly twice as slow to start a trivial program, and that difference is a few milliseconds. What *is* true and defensible: there is no per-operation penalty. The overhead is a fixed startup cost of a few milliseconds, not a tax that grows with the work. At 61 microseconds of steady-state work, portability is costing nothing once running.

8.1.1 The honest sentence to use

Say this

"A Krate app starts in about 11 milliseconds — roughly 6 milliseconds behind the same program compiled natively, which nobody can perceive — and it is about 19 times smaller. After it starts there is no per-operation cost at all."

8.1.2 Where Krate genuinely wins on speed

Against Electron, which is the real competition for cross-platform apps: Electron apps typically take hundreds of milliseconds to start and occupy 80–200 MB. Krate starts in milliseconds at 15–30 KB. That comparison is where the word "faster" is honest.

8.2 Size

Thing	Size	Ratio vs Krate
Krate app (typical published)	15–30 KB	1×
Same program as a native binary	336 KB	19× larger
Typical Electron app	80–200 MB	≈5,000× larger
The Krate runtime itself (installed once)	21 MB	—

The honest caveat about the 21 MB

The apps are tiny, but the runtime you install once is 21 MB. That is a fair thing for a sceptic to point at. The answer: it is installed once and shared by every app, so the tenth app you receive costs 20 KB, not another 21 MB. We have not yet audited what makes up those 21 MB — turning off the speech feature only saved 1 MB, so the common assumption that speech is the bulk is wrong.

8.3 Reliability

Measure	Value on 11 Aug 2026
Automated tests, all passing	1,141
Example apps in the repository	39
Example apps passing every check	32 of 32
Bugs recorded on the public board	96
Bugs fixed	82
Releases shipped	36
Platforms built and verified per release	6

The six platforms are: macOS on Apple silicon, macOS on Intel, Windows on Intel, Windows on ARM, Linux on Intel, and Linux on ARM. After each release is published, a separate job downloads the published files on three operating systems and verifies checksums, signatures, and that an app actually builds and runs.

8.4 The benchmark, stated honestly

There is a benchmark of how often the AI produces a usable app. Its last recorded run was 5 August 2026, and it scored **0 out of 5** on authored apps under a strict "did it do what was asked" test.

This number is old and must be re-run

Dozens of fixes have landed since that run, and apps written after it (a tip calculator, a JSON formatter, a Markdown table tool, a Pomodoro timer) do pass. But **we have not re-measured**, so there is no current number. If someone asks "what is your success rate", the correct answer is: "the last honest measurement was before a lot of fixes and it was bad; re-running it is on the list, and I will not quote a number I have not measured."

That is uncomfortable, and it is deliberately in this document, because quoting a number you cannot defend is worse than admitting you have not measured it yet.

Chapter 9

How It Was Built

9.1 It was planned as an eight-phase, two-year project

Krate was not built by improvising. Before the code, there were eight written phase plans, each with a one-sentence objective, a duration, and explicit success criteria. They are still in the repository. Here they are, with the duration each was originally given:

	Originally planned	The objective, in its own words
0	Weeks 1–4	Get the project bones set up so real work can start
1	Months 2–3	Prove one `.wasm` runs identically on three desktop hosts
2	4–8 weeks	Ship the first useful cross-platform CLI app
3	6–10 weeks	First GUI app running natively on Windows, macOS and Linux
4	Months 11–14	The same `.krate` runs on iOS and Android without source changes
5	Months 15–18	A developer gets a working GUI app in under 60 seconds
6	Months 19–22	Users discover, install, update, and sign in across devices
7	Months 23–24	Launch quality, and ship v1.0 in public

The single most impressive fact in this document

Phase 4 — "the same `.krate` runs on iOS and Android without source changes" — was scheduled for **months 11 to 14**.
It ran on a real Android phone on 9 August 2026 and on a real iPhone on 10 August 2026, in month five.
The plan was not abandoned; it was beaten. Phases 0 through 3 are done, and Phase 4's core proof landed roughly nine months ahead of its own schedule.

9.2 The timeline of what actually happened

When	What happened
April 2026	Work begins, under the name Layer36. At this point there is no evidence that any of it is possible.
24 June 2026	The status record begins. Three real command-line apps (`krate-clock`, `krate-cat`, `krate-curl`) run through the runtime.
2 July 2026	Phase 2 formally closes out. The plan is amended rather than followed blindly.
4 July 2026	**The milestone.** All three desktop operating systems open a real native window from the same portable file, proven in one green CI run.
July 2026	Renamed from Layer36 to Krate — the previous name collided with other AI startups. The two-paths idea (receive vs. make) is set.
3 August 2026	The current shipping repository's first commit.
5 August 2026	Krate Cloud goes live; the first app is published from a terminal.
9 August 2026	The Android player runs a real published app from a tapped link.
10 August 2026	The iOS player runs on a real iPhone, under Pulley. GPU rendering added on Metal.
11 August 2026	Seven numbered bugs (K-091 to K-097) fixed in one day; v0.1.10 and v0.1.11 ship.

Why 4 July 2026 is the date that mattered

Everything before it was a bet. One file, compiled once, opening a real native window on macOS, Windows and Linux — with a robot clicking the button on Linux and the app noticing — is the moment the central premise stopped being a hypothesis. Every later achievement (the phones, the cloud, the AI loop) is built on that one proof.

What "289 commits" actually means

The repository you can see today was started on 3 August 2026, so its 289 commits span eight days. That is the *current* repository, not the project. The work began in April under the name Layer36; the status record runs from 24 June; the phase plans are older still; and the three-operating-system proof was 4 July — all before the first commit here.
So when somebody points at the commit rate and implies the work is thin, the accurate answer is: this is the shipping repository of a project that started in April, and it moves fast because four phases of design and proof were finished before any of this code was written.

9.3 What was genuinely uncertain

It is worth being able to say what was *not* known in April, because the answer is: almost all of it.

- **Whether one binary could really draw a native window on three operating systems.** That was Phase 3's entire purpose, and it was a real question — each system has a completely different windowing model.

- **Whether apps could be written without Rust's standard library.** Krate refuses operating-system calls, but ordinary Rust code pulls them in invisibly. This was unresolved until the `no_std` guest work, and until it was solved, three out of eight AI-written apps failed for this reason alone.
- **Whether an AI could hit an API it had never seen.** There was no reason to assume yes. The answer turned out to depend less on the model than on having a checker that could say exactly what was wrong.
- **Whether the phones would work at all.** Phase 4 was over a year away in the plan and could have failed outright — iOS forbids just-in-time compilation, which is how WebAssembly is normally made fast.

That last one deserves its own note, because it is the best engineering answer in the project.

The iPhone problem, and the answer

Apple does not permit an app to generate machine code while running. That is exactly how a fast WebAssembly runtime normally works, so on paper Krate could not run on an iPhone at all.
The answer was Pulley — a portable interpreter — instead of just-in-time compilation. It was then verified to produce **pixel-identical** output to the compiled path. The constraint was real, the workaround was real, and it was checked rather than assumed.

9.4 The method: a public bug board

Every defect found gets a number, an owner, a severity, and — crucially — *evidence*: the exact command and its output, not a description. 96 entries, 82 fixed.

The board also classifies who can fix each bug, which turns out to be the most useful part:

Class	Count	Meaning
`our-code`	67	An ordinary defect in Krate
`example-bug`	10	Our example apps teach the bug, so every generated app inherits it
`runtime-hole`	7	The runtime cannot do it; no prompt helps
`environment`	7	This machine, not the product
`teaching-hole`	4	The runtime can do it and we never told the AI

Why `example-bug` is the highest-leverage class

An AI writing a Krate app reads our example apps to learn how. So a bug in an example is not one bug — it is a bug in every app anyone generates afterwards. Ten entries are of this kind, and fixing one of them fixes apps that have not been written yet.

9.5 Five real problems, and what they taught

These are worth knowing in detail, because they are the honest story of how the product got good, and they answer "how do I know this is solid?"

9.5.1 The 90-gigabyte memory leak

A canvas game grew to 46 GB of memory when running in a window. The macOS drawing code was creating a fresh image object every single frame and never releasing them. Fixed by reusing one image view and draining the memory pool each frame. Memory is now flat at about 80 MB.

Lesson: a bug that only appears in one mode (windowed, not headless) will not be found by tests that never open a window.

9.5.2 Every Windows release shipped a broken zip file

The ZIP file format specification requires forward slashes in paths. Windows' own `Compress-Archive` writes backslashes. Every Windows release we had ever shipped was technically malformed. It was found only when we built a job that downloads the published files and checks them.

Lesson: verifying what you published is different from verifying what you built.

9.5.3 The app that was impossible to build

An outside reviewer asked for a folder-tidying app and found it could not exist: no app could name a folder, and no dialog existed to pick one, so the AI reached for unrestricted filesystem access instead. He was right. The answer was to invent "the pick is the grant" (Chapter 3), add it to the runtime, and ship the app he described.

Lesson: when a reviewer says something is impossible, check whether they are right before defending the design.

9.5.4 The product was slow, and it was not the runtime

For months, a Krate app took 74 milliseconds to launch while the runtime itself needed only 3.3. The other 91% was Krate blocking every launch on a telemetry message to our server. Fixed by writing the event to a file and letting a separate process send it. Launches dropped from 74 ms to 6 ms.

Lesson: measure the whole path a person experiences, not the component you are proud of.

9.5.5 Apps that closed their own windows

Fifteen example apps ended their event loop after a fixed number of rounds — about thirty seconds of use — so the window vanished while somebody was using it. Including the app the AI is told to copy first.

Lesson: an example app is documentation. A bug in it is a bug in everything written afterwards.

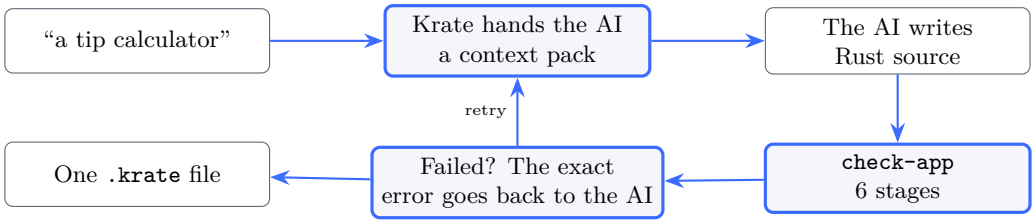
Chapter 10

How the AI Part Works

10.1 Krate does not have its own AI

This is the single most misunderstood point, and the answer is a strength, not a weakness.

Krate does not train, host, or pay for any AI model. It drives whichever AI coding tool you *already have installed* — Claude Code, Codex, Gemini, Copilot, or Grok. If you pay one of them, you pay nothing extra to Krate for authoring.



10.2 The context pack: how the AI knows what to write

An AI cannot guess an API it has never seen. So Krate generates a document for it, freshly, from the actual source code: every callable function, every capability name, the interface definitions, the discipline rules, and an index of example apps by shape.

Because it is generated from the same source the runtime compiles, it cannot drift out of date — a class of failure that would otherwise silently teach models an API that no longer exists. A test enforces this.

10.3 The loop that makes it work

The AI writes code. `check-app` judges it and, on failure, returns the specific stage and a suggested fix. That goes back to the AI, which tries again. The person sees none of this — they see a progress line and then a file.

Why this matters more than the model

Any model writes broken code sometimes. What makes output trustworthy is not a better model — it is having a machine that can tell, definitively, whether the code is acceptable, and can say exactly why not. That checker is the actual product moat.

10.4 Honest limitations of the AI path

- A build takes **5 to 12 minutes**, most of it compiling Rust.

- It sometimes needs a second attempt.
- The success rate has not been re-measured since 5 August (see Chapter 5).
- Authoring depends on other companies' tools, which can change. Supporting five providers is the mitigation.

Chapter 11

The Cloud and the Business

11.1 What Krate Cloud is today

Krate Cloud went live on 5 August 2026. It is a Cloudflare Worker with object storage behind it, and it does nine things:

Route	What it does
`POST /publish`	Takes a `.krate`, stores it, returns a URL
`GET /a/<id>`	Serves the app so `krate run <url>` works
`GET /apps`	The public gallery
`GET /shot/<id>`	The app's screenshot
`POST /auth/start`	Begins GitHub device-flow sign-in
`GET /auth/poll`	Completes it
`POST /usage`	Anonymous usage counting
`GET /stats`	Aggregate numbers
`GET /health`	Liveness

Twelve apps are published on it. Publishing is free and requires no account to *run* anything — a link is enough.

One design decision worth stealing

Apps are **content-addressed**: the storage key is a hash of the file itself, so the same app always lands at the same URL. There is nothing to overwrite and nothing to look up. It also means a URL is a proof of contents — what you fetch is exactly what was published.

11.2 The business model, stated plainly

Layer	Status
The runtime	Free and open source, forever. This is the adoption engine, not a product.
Authoring	Free. You bring an AI tool you already pay for; Krate takes nothing.
Publishing	Free today, and free for individuals in any plausible future.
Revenue, later	Private galleries, team distribution, usage above a free tier, and support for organisations that need to distribute internal tools.

Nothing is priced yet, and that is deliberate

There is no pricing page, no paid tier, and no revenue. Charging before the adoption loop works would optimise the wrong thing — and the honest state is that gate G5 (ten people outside this machine making an app and sending it to someone) is not met.
If an investor asks "what's the business model", the answer is the table above plus that sentence. Inventing a revenue projection for a product with no users is the fastest way to lose a technical investor's confidence.

11.3 Why the free runtime is not a giveaway

The comparison is Docker, and the parallel is close enough to use out loud: the container runtime is free and everywhere, and the money is in the registry, the team features, and the enterprise controls. Nobody would have adopted Docker if the runtime cost money, and nobody would adopt Krate if receiving an app required a subscription.

In everyday terms

The PDF reader was free. Adobe made its money on everything around it. Making the reader free is what made the format universal, and the format being universal is what made the rest worth anything.

11.4 The market, in the only terms that are checkable

Avoid invented totals. The defensible framing is a sequence of facts:

1. AI coding tools went from niche to mainstream in about 18 months, and the people using them increasingly cannot code.
2. Every app so produced needs somewhere to go, and the existing options (app stores, web hosting, Electron) were built for companies shipping to millions.
3. Software with an audience of one — a tool for your mum's invoices, a tracker for a team of four — has no distribution channel at all.
4. That gap is what Krate fills, and it did not exist two years ago.

The sentence to use

"I'm not going to give you a market size I made up. What I can tell you is that the number of people making software went up by an order of magnitude, the number of places to put it went up by zero, and that gap is new."

Chapter 12

How Krate Compares

12.1 The comparison table

	Krate	Electron	Flutter	Native	Web app
Size of one app	15–30 KB	80–200 MB	10–40 MB	1–20 MB	—
One file, all systems	yes	rebuild	rebuild	no	n/a
Works offline as a file	yes	yes	yes	yes	no
Permission wall enforced	**yes**	no	no	OS-level	browser
Built for AI authorship	**yes**	no	no	no	no
Store approval to share	**none**	none	stores	stores	none

12.2 The gap nobody else fills

Every existing option makes you choose:

- **Small and safe**, but not really yours — a web app.
- **Real and installable**, but huge and unrestricted — Electron or native.

Krate is a real file you own, tiny, and sandboxed by default. That combination is the position.

12.3 What about other WebAssembly runtimes?

Wasmtime, Wasmer and WasmEdge are excellent engines — Krate uses wasmtime internally. But they are engines for servers and embedding, not products for people. None of them gives you a double-clickable app file, a permission wall in plain words, a window on three operating systems, an AI authoring loop, or a publish-and-share link.

In everyday terms

Wasmtime is an engine. Krate is the car.

Chapter 13

Where Krate Is Honestly Not Ready

Volunteering this is the cheapest credibility available, and every item here is something a sceptic would otherwise find.

1. **Smartphone players are days old.** Android and iOS both run real apps, but responsiveness on iPhone is still being tuned. They are not in any app store.
2. **Not hardened against hostile code.** The wall stops honest apps from overreaching. It is not an adversarial security boundary and is never claimed as one.
3. **Windows shows a SmartScreen warning.** macOS is fully signed and notarised; Windows needs a certificate we have not bought.
4. **AI authoring takes 5–12 minutes** and sometimes needs a retry.
5. **No heavy 3D or video.**
6. **The authoring success rate has not been re-measured** since 5 August 2026.
7. **The 21 MB runtime has not been audited** for what makes it that size.
8. **One engineer.** The bug board, the goals file and the plan documents exist partly to make the work legible to someone else.

Chapter 14

What Happens Next

14.1 The gates that define “version 1”

	Gate	Status on 11 Aug 2026
G1	A public, honest benchmark number	Harness exists; needs re-running
G2	No blocker on the bug board	82 of 96 fixed
G3	Usability enforced, not hoped for	Done — stage 6 of check-app
G4	The outsider path works cold	Run once, defects found and fixed
G5	Ten real people have made and sent an app	Not yet

G5 is the only one that cannot be engineered around.

14.2 The engineering queue

1. Re-run the authoring benchmark and publish the number.
2. Finish smartphone responsiveness; port GPU rendering to Android.
3. Windows code-signing certificate.
4. Audit the 21 MB runtime.

14.3 One that just came off the list, and what it teaches

“Make an arbitrary new app look good by default” was on this queue. It was fixed on 11 August 2026, and the shape of the bug is worth keeping, because it is a kind that will happen again.

The runtime had grown proper drawing tools — rounded rectangles, soft shadows, gradients with stops, and font weights. But the document the AI reads before writing an app still contained older advice, in prose, telling it to fake a rounded card by drawing a rectangle and four circles at the corners. Nine of our own example apps still did it that way, and the AI learns by copying those.

The result was that Krate could draw beautifully and generated apps looked dated anyway. It was visible: in one app every empty checkbox and text field rendered as a *broken dotted box*, because the hand-built outline drew its corners as single pixels.

The fix was to rewrite the guidance around the tools that exist and convert all nine apps. Then it was tested the only way that counts — by asking for an app nobody had built before, a wedding seating-chart planner:

In the freshly generated app	Times used
`drop_shadow_round_rect` (never taught before)	2
`draw_text_styled` (never taught before)	2
`measure_text_styled` (never taught before)	2
`fill_round_rect` / `stroke_round_rect`	2
The old four-circle corner hack	**0**

It produced a serif display heading, a gold-on-charcoal palette it chose for a wedding, a floor plan of round tables with seat dots showing who is seated, and a side panel of unseated guests — in a 29,762-byte file.

The lesson worth repeating

When a capability ships, the work is not finished until the *prose* the AI reads changes and the examples stop demonstrating the old way. A function list that a model never reads is not teaching. This is why the bug board has a `teaching-hole` class at all.

Chapter 15

Questions and Answers

This chapter is written so you can answer anything, in the questioner's own language, without preparation.

15.1 From a normal person

"What is Krate, in one sentence?"
An AI writes you a small app, and Krate turns it into one file that opens on any computer and tells you what it wants before it runs.

"Why should I care?"
Because you can finally make a small tool for yourself — a tracker, a calculator, something for your mum's invoices — and actually send it to someone without them needing to install anything.

"Is it safe to open a file a stranger made?"
Safer than any normal program, because before it runs you see what it wants and can refuse. An app that gets no permissions can genuinely do nothing but draw in its own window.

"Do I need to know how to code?"
No. You type what you want in ordinary words.

"Does it cost anything?"
The runtime is free and open source, and publishing is free. You need one AI coding tool, which most people who want this already pay for.

15.2 From a developer

"Why WebAssembly rather than a scripting language?"
Because Wasm is compiled (fast), portable (one file everywhere), and sandboxed at the boundary rather than by convention. A Python script cannot make the permission promise; a Wasm component can.

"What is actually in the .krate file?"
A zip containing the compiled WebAssembly component, the manifest, and optionally assets and source.

"How is the sandbox enforced?"
The component can only call functions the runtime hands it. Every call passes a guard that checks it against the session's granted capabilities. Filesystem paths resolve inside the app's own directory with symlink and containment checks; absolute paths and `~` are refused at parse time.

"What is the performance cost?"
About 6 milliseconds more than native to start, and 61 microseconds of steady-state work per invocation — no per-operation tax. Numbers in Chapter 5.

"Why Rust for the apps?"
Because it compiles to small, fast Wasm with no runtime of its own. The person never writes it

— the AI does.

“What is ‘no_std’ and why does it matter?”
Rust’s standard library assumes an operating system, and pulling it in drags along calls Krate refuses. Apps are written without it, so they import only Krate’s own interfaces. `check-app` verifies this.

“What happens when my app needs something Krate does not have?”
Today, you cannot do it. That is a real limit, tracked on the board as `runtime-hole`, and those are the entries we prioritise.

15.3 The hard technical questions

These are the ones that come from people who know the field. Each answer is short on purpose — you should be able to say it out loud.

“Why not just use WASI? It already standardises this.”
WASI standardises an app’s access to an operating system — files, sockets, clocks. Krate deliberately does not give apps an operating system; it gives them a capability-checked set of interfaces and refuses `wasi:*` entirely. A Krate app importing WASI is a build failure, by design. Different goal, not a worse implementation of the same one.

“Isn’t this just Java applets again?”
The comparison is fair and the differences are the ones that killed applets: applets needed a browser plugin, ran with a security model people could not see or reason about, and were separate from the ordinary way you open a file. A `.krate` is a file you double-click, its permissions are four lines of plain English shown before it runs, and it does not need a browser at all.

“Sandboxing at the interface boundary — what about side channels, Spectre, resource exhaustion?”
Not defended against, and we say so rather than let someone find out. The wall stops honest apps from overreaching; it is not an adversarial security boundary. What *is* bounded today: bundle sizes, decompressed entry sizes (the zip-bomb case), and path escapes. What is not: a deliberately hostile app burning CPU or exploiting the processor itself.

“What is the actual startup cost breakdown?”
3.3 ms to compile and instantiate the component, 61 microseconds of steady-state work per invocation, and about 11.3 ms end to end for a whole process. Until 11 August 2026 it was 73.9 ms, and 91% of that was Krate blocking on a telemetry request — not the runtime.

“Rust for app authors is a strange choice. Why not something easier?”
Because a person never writes it — an AI does — so the usual objection (the learning curve) does not apply, while the benefits do: no garbage collector, no language runtime to ship, and components in the tens of kilobytes. If a person wants to write one by hand, they can; the SDK is public.

“How do you handle versioning when the interfaces change?”
Worlds are versioned (`krate:app/gui@0.2.0`) and a tool called `check-uapi-freeze-lock` runs in CI to stop an interface being changed silently. When an app is too old for a runtime, it gets a plain sentence rather than a crash.

“What happens when the runtime has a security bug?”
The same thing that happens to a browser: it ships a new version. That is the trade of a shared runtime — one place to fix, but everyone must update. Releases are signed and notarised on macOS, and the release gate verifies the published artefacts on three operating systems.

“Your example apps are hand-tuned. Does a real generated app look like that?”
That was a fair criticism and it was a real bug. As of 11–12 August 2026, the authoring pack’s design guidance was still teaching a workaround for primitives that had shipped, so generated apps looked dated while our showcase apps did not. It is fixed, and it was verified by generating an app type nobody had built — a wedding seating planner — and checking which drawing calls it reached for. See Chapter 12.

“Nine hundred thousand lines in four months. Be serious.”
618,821 of those lines are machine-generated bindings. The hand-written figure is 112,831 lines of Rust plus 46,910 of documentation. Quote the smaller number — it is the honest one, it is still a serious body of systems code, and it does not fall apart when someone checks. See Section 4.2.

15.4 From an investor

“What is the market?”
Every person who will make software with AI and needs somewhere for it to go. App stores were built for companies shipping to millions, not for a tool with an audience of one.

“Why now?”
Because the making of software got solved in the last 18 months and the delivering of it did not change at all.

“What is defensible here?”
Not the WebAssembly engine — that is open source. What is hard to copy is the verification loop (a machine that can tell whether AI-written code is acceptable and why not), the capability model with plain-language consent, and the six-platform delivery pipeline that verifies what it published.

“How much are you raising and what for?”
Approximately $750,000 on a SAFE. It funds roughly 18 months: the founder full-time, the immigration path to the US, code-signing certificates, cloud costs, and one senior engineer once the adoption loop is proven.

“How will the money be spent?”

Use	Share
Founder salary, 18 months full-time	largest single line
One senior engineer, after adoption is proven	second largest
Immigration and relocation to the US	fixed cost
Code signing (Apple, Microsoft) and cloud	small, unavoidable

“What is the business model?”
The runtime is free and open source; that is the adoption engine. Revenue comes later from the cloud: private galleries, team distribution, and usage beyond a free tier. Nothing is priced yet, deliberately — adoption first.

“What are the traction numbers?”
Small, real, and measured. The cloud counts anonymous usage, and as of 12 August 2026 it reports:

Measure (from `hub.krate.tech/stats`)	Value
Distinct installs, 90 days	246
App opens, 5 days of records	4,187
Failed opens, same period	425
Open success rate	90.8%
Apps made	12 (all AI-authored)
Apps published to the cloud	2
Apps in the public gallery	12

State the caveat before they ask for it

Installs include continuous-integration machines and our own development machines — they are not 246 people. The number that matters is the one that is still nearly zero: **2 apps published by anyone, and no outside person has yet made an app and sent it to a friend.** That is gate G5, and it is unmet.
The engineering is far ahead of the distribution. Say that plainly; it is the correct read and pretending otherwise fails instantly.

One number in that table is worth watching for a different reason: a 90.8% open success rate means roughly one in eleven attempts to open an app failed. That is a product-quality signal we get for free, and it is the kind of thing worth fixing before chasing more installs.

What happened when we looked — a lesson in measurement

That 9.2% was found by reading the live statistics while writing this document. Nobody had reported it. And when we went to fix it, the telemetry could say only *that* an open failed, never *why* — so the number was alarming and unactionable at the same time.
Failed opens now carry one word from a fixed list: `refused`, `not-found`, `bad-bundle`, `bad-manifest`, `version-too-old`, `no-window`, `app-failed`, `other`. A closed list, not free text, so it can never carry a file path or an app name.
The first finding was definitional and it changes the number. `refused` — the permission wall turning an app away — exits with code 5, and code 5 was being counted as a failed open. Some share of those 425 failures was *the product working exactly as designed.* The true defect rate is lower than 9.2%; how much lower needs a week of data.
If someone asks you about this number, that is the answer: we found it ourselves, we could not explain it, so we built the thing that explains it before quoting a cause.

15.5 The hard questions

"289 commits in eight days. Is this AI-generated slop?"
The defence is not volume, it is verification: 1,141 automated tests, a release pipeline that downloads its own published files on three operating systems and re-verifies them, and a public bug board with 96 numbered entries where the failures are documented with evidence. Judge the work by what it survives, not by how fast it appeared.

"What stops Apple, Microsoft or Google from doing this?"
Nothing — and if one of them does it, the thesis was right. What we have is a working implementation, a public track record of fixing what real people report, and no incentive to protect an existing app store.

"Why hasn't anyone done this already?"
The WebAssembly component model only became usable recently, and the demand only exists because AI can now write apps for non-programmers. Both halves are about two years old.

"Your own benchmark said 0 out of 5."
It did, on 5 August 2026, under a deliberately strict test. That number is in this document rather than hidden, and it drove the fixes that came after. It has not been re-run, so we do not quote a better one.

"You are one person. What happens if you are hit by a bus?"
That is a real risk and the reason for the documentation discipline: the bug board, the goals file, the plan documents, and this guide exist so the work is legible to somebody else. The first engineering hire reduces it further.

"Is the permission wall real security?"
It is real enforcement against honest apps that overreach — the check happens at the boundary,

on every call. It is not hardened against deliberately hostile code, and we say so everywhere rather than let someone discover it.

“Why would anyone trust an app a stranger’s AI wrote?”
They would not, and that is the point of the wall. You do not have to trust the app. You only have to read four lines of plain English and decide whether to allow them.

15.6 Questions in the style of Y Combinator

“What do you do?”
We make AI-written apps shareable. One small file, opens on any computer, tells you what it wants first.

“Who wants this urgently?”
The person who just had an AI build them something useful and cannot give it to anyone.

“What have you learned from users?”
That the failures are never where you expect. A tester hit a compiler error and stopped — which produced the two-paths design. A friend on Windows found a generated game drawn off the screen — which exposed that 21 of our own examples had the same defect and our own quality gate could not see it. Both are in this document because both changed the product.

“What is the one metric that matters?”
Ten people who are not us making an app and sending it to someone. That is gate G5, and it is not met yet.

Chapter 16

Explaining Krate Out Loud

Everything so far is for you to *know*. This chapter is for you to *say*. Each version is complete on its own — pick by who is asking and how long they have.

16.1 Ten seconds

The one-liner

"AI can write you an app in a minute, but you still can't send it to anyone. We make it one small file that opens on any computer and tells you what it wants before it runs."

16.2 Sixty seconds

"You know how if you make something useful with AI — a little tracker, a calculator for your work — you basically can't give it to anyone? They'd need the right programming languages installed, or you'd have to put it on a website and pay for it forever, or it arrives as a 200-megabyte download that Windows warns them about.

Krate makes it one file, about 20 kilobytes, that opens by double-click on Mac, Windows and Linux. And because nobody should run a program a stranger's AI wrote, it tells you what it wants first — in plain words, like 'save files in its own private folder, never yours' — and you can say no and it still opens.

It works today. Thirty-nine apps, six platforms, and it runs on phones too."

16.3 Three minutes, for someone technical

Lead with the boundary, because that is the part that is actually novel:

"It's a WebAssembly component runtime with a capability wall enforced at the dispatch boundary — every host call checks the session's grants, and apps import only `krate:*`, never `wasi:*`, which is verified on the compiled artefact rather than trusted.

The interesting bit is what we call 'the pick is the grant'. A folder-tidying app cannot name a folder — there is no capability that would let it. It asks the person to pick one, and the pick becomes a token scoped to that run. The example app ships with zero filesystem capabilities declared.

The other half is the authoring loop: whatever AI you already pay for writes the app, and a six-stage checker decides whether the output is acceptable — including a stage that opens the window, resizes it, clicks it, and watches whether it closes itself. That checker is the actual product, more than the runtime is."

16.4 The questions that follow, and the short answers

They ask	You say
"Is it faster than a normal app?"	"No — about 6 milliseconds slower to start, which nobody can feel. It's 19 times smaller, and once running there's no per-operation cost."
"How big is the runtime?"	"21 MB, installed once. The tenth app you receive costs 20 KB, not another 21."
"Who's it for?"	"The person who just had an AI build them something and can't give it to anyone."
"How many users?"	"Testers, not users. The engineering is ahead of the distribution — that's the honest position."
"Is it secure?"	"It's real enforcement against apps that overreach. It's not hardened against deliberately hostile code, and I'd rather say that than have you find it."
"What's stopping Apple doing this?"	"Nothing. If they do, the thesis was right. We have a working implementation and no app store to protect."

16.5 Three rules for talking about it

1. **Name the baseline on every comparison.** "Smaller" is meaningless; "19 times smaller than the same program compiled natively, thousands of times smaller than Electron" is checkable.

2. **Volunteer the weakness before it is found.** Saying "the benchmark said 0 of 5 and I haven't re-run it" buys more credibility than any number you could quote instead.

3. **Never quote a number you haven't measured.** Every figure in this document has a date attached for exactly this reason. If it is older than the last big change, say so.

Chapter 17

A Worked Case Study: Making Generated Apps Look Good

This chapter follows one problem from complaint to fix, because it shows how the whole system — runtime, teaching, examples, gate — has to move together, and because it is a good story to tell.

17.1 The complaint

Two pieces of feedback, days apart. A friend testing on Windows: a generated game's character and ground were off screen. And, more generally, that Krate apps looked basic — "like from a few years back".

17.2 What was actually wrong (three separate causes)

17.2.1 Cause 1: apps ignored the window

Twenty-one of our own 32 example apps drew at a fixed size. On a window that was not exactly that size, content fell outside it. The gate could not see it, because it measured whether the *canvas* followed the window, not whether the picture did.

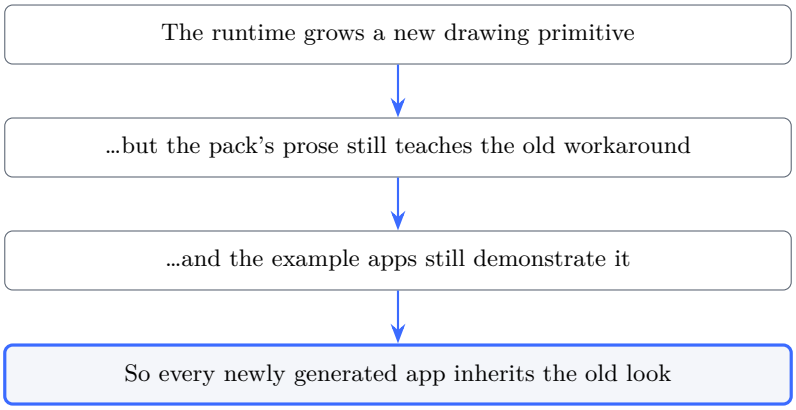
Fixed by `set-design-size`: the app declares its own coordinate system, and the runtime scales it uniformly and centres it. The gate was rewritten to compare what a person actually sees.

17.2.2 Cause 2: the teaching lagged the runtime

The renderer had grown rounded rectangles, soft shadows, gradients with stops, and font weights. But the document the AI reads still said, in prose, to fake a rounded card with a rectangle and four circles — contradicting the function list further down the same file. **A model follows the prose.**

17.2.3 Cause 3: the examples taught it too

Nine example apps still hand-built rounded corners. An example app is documentation; the AI copies it. One of them drew its outlines with 1-pixel corner dots, so every empty checkbox and text field in that app rendered as a *broken dotted box*.



17.3 How it was verified

Not by reading the code — by generating an app of a type nobody had built (a wedding seating planner) and inspecting which drawing calls it reached for. A `.krate` bundle ships its own source, so this is checkable from the outside:

Call	Before	After
`drop_shadow_round_rect`	0	2
`draw_text_styled`	0	2
`measure_text_styled`	0	2
`fill_round_rect` / `stroke_round_rect`	0	2
The four-circle corner workaround	used	**0**

17.4 The part that is still open

Then the layout guidance was added — fill the window, size grid cells from the area, keep one elastic region — and regenerating the same app showed the tables filling the room properly. But it also introduced a new defect: the guest names now overlapped the chairs, because the app measured from the table's radius while the chairs sat further out.

That was fixed with a further rule (measure from the outermost edge, not the shape). But regenerating again produced a *third* layout that cleared the overlap and was less dense again.

The honest conclusion, filed as K-099

The AI trades density against overlap between runs, and prose guidance cannot hold both. This needs a machine check, the way the resize bug did.

An edge-band check was written for it and **thrown away**: measured against six real screenshots it scored every app at 3–6%, including one with an obviously dead region, because real apps put a full-width panel near each edge so no complete row is ever empty. A gate that detects nothing while claiming to is worse than no gate. The evidence sits in the bug entry so the next attempt starts from a region measure instead.

17.5 What this case study is for

Three lessons, all of which generalise:

1. **A shipped capability is not a delivered one.** It reaches users only when the teaching and the examples change too.

2. **Test the cold path, not the code path.** The bug was found by generating an unfamiliar app, not by reading source.

3. **Prose guidance drifts; gates hold.** Anything you cannot encode as a check will regress the next time the model runs.

Chapter 18

Glossary

Term	Plain meaning
WebAssembly (Wasm)	A portable form of compiled code that runs anywhere and can do nothing unless permitted
Runtime	The program that runs other programs; Krate is one
Component	One compiled Wasm app plus a description of what it needs
Capability	One specific power an app may ask for, e.g. “open a window”
Manifest	The file where an app declares what it wants, and why
Sandbox	A boundary an app cannot reach outside of
`.krate`	Krate’s app file: a zip with the component, manifest and assets
Native	Compiled for one specific operating system and chip
Electron	A way of shipping web apps as desktop apps, by including a browser
Headless	Running without a visible window, for automated checks
`no_std`	Writing Rust without its standard library, so no hidden OS calls appear
Component model	The WebAssembly standard for typed interfaces between a host and a guest
WIT	The language those interfaces are written in
Guest	The app running inside the runtime
Host	The runtime providing capabilities to the guest

Chapter 19

Where Everything Lives

What	Where
The product	https://krate.tech
The source	https://github.com/incyashraj/krate
The cloud gallery	https://krate.tech/cloud
The bug board	`BUGS.md` in the repository
Direction and gates	`GOALS.md`
Verified claims for outreach	`Invest/OUTREACH_TRUTH.md`
The development queue	`Plan/krate-dev-queue-2026-08.md`
This document	`Doc/krate-complete-guide.tex`

Keeping this document true

Every figure here carries the date it was measured. When the numbers change, change them here and change the date. A guide that quietly goes stale is worse than no guide, because it makes you confident about something that is no longer true.